SIMATS ENGINEERING

CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1709

Name	**Shaik Mahaboob Basha**
Register Number	**192365045**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Code of Conduct:

I Shaik Mahaboob Basha (192365045)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Shaik Mahaboob Basha

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).
(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1Q)

Objective:

To implement and evaluate a **Decision Tree Classifier** on the Iris dataset using Python and Scikit-learn.

(a) Load, Pre-process and Split the Dataset

The **Iris dataset** contains 150 flower samples with four input features:

- Sepal length
- Sepal width
- Petal length
- Petal width

The target has three classes:

- Setosa
- Versicolor
- Virginica

```
import pandas as pd
import numpy as np

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
iris = load_iris()
df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)
df["species"] = iris.target
print("First 5 rows:")
print(df.head())
print("\nData Types:")
print(df.dtypes)
print("\nMissing Values:")
print(df.isnull().sum())
X = df.drop("species", axis=1)
y = df["species"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
```

```
    random_state=0,
    stratify=y
)

print("\nTraining samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
= RESTART: E:\ai\lab\exp1\test\41a.py
First 5 rows:
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  species
0                5.1               3.5  ...               0.2        0
1                4.9               3.0  ...               0.2        0
2                4.7               3.2  ...               0.2        0
3                4.6               3.1  ...               0.2        0
4                5.0               3.6  ...               0.2        0

[5 rows x 5 columns]

Data Types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
species                int32
dtype: object

Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
species              0
dtype: int64

Training samples: 105
Testing samples: 45
5
.. 5.1     3.5     1.4     0.2     0
.. 4.9     3.0     1.4     0.2     0
.. 4.7     3.2     1.3     0.2     0
.. 4.6     3.1     1.5     0.2     0
.. 5.0     3.6     1.4     0.2     0
```

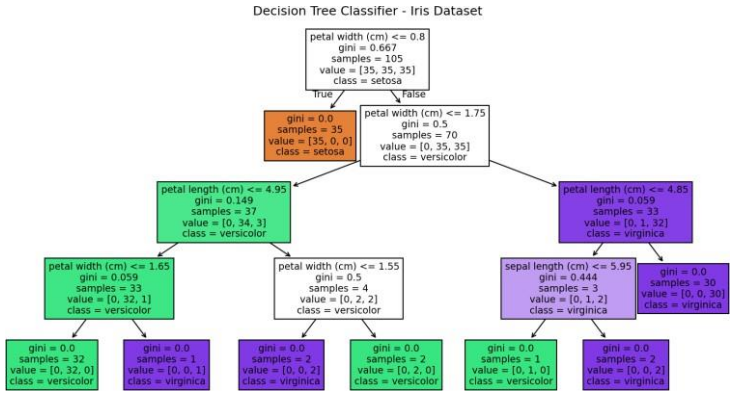
B)

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text, plot_tree
import matplotlib.pyplot as plt
iris = load_iris()
df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)
df["species"] = iris.target
X = df.drop("species", axis=1)
y = df["species"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=0,
    stratify=y
)
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=0
)
model.fit(X_train, y_train)
tree_rules = export_text(
    model,
    feature_names=list(X.columns)
)

print("Decision Tree Structure:")
print(tree_rules)
plt.figure(figsize=(15, 8))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=iris.target_names,
    filled=True
)

plt.title("Decision Tree Classifier - Iris Dataset")
plt.show()
```



C)

```
import pandas as pd

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
iris = load_iris()

df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)
df["species"] = iris.target

X = df.drop("species", axis=1)
y = df["species"]
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=0,
    stratify=y
)

model = DecisionTreeClassifier(
    criterion="gini",
    random_state=0
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)
```

```
Accuracy: 0.9777777777777777

Confusion Matrix:
[[15  0  0]
 [ 0 15  0]
 [ 0  1 14]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       0.94      1.00      0.97        15
   virginica       1.00      0.93      0.97        15

    accuracy                           0.98        45
   macro avg       0.98      0.98      0.98        45
weighted avg       0.98      0.98      0.98        45
```

2Q)

Objective

Implement Q-Learning for a 4×4 Grid World, learn the optimal policy, observe how the Q-table changes during training, and plot cumulative reward per episode.

(a) Environment Definition

We use:

- Grid: 4 × 4
- Start state: (0,0)
- Goal state: (3,3)
- Obstacles: (1,1) and (2,2)
- Actions: Up, Down, Left, Right
- Rewards:
  - Goal → +10
  - Normal step → -1
  - Obstacle → -5
- Initial Q-table: all zeros
- Learning rate α: 0.1
- Discount factor γ: 0.9
- Initial exploration rate ε: 1.0
- Minimum ε: 0.05
- Episodes: 100

The Q-learning update equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma a' \max Q(s',a') - Q(s,a)]$$

code:

```
import numpy as np
size = 4
start, goal = (0, 0), (3, 3)
obstacles = {(1, 1), (2, 2)}

actions = ["Up", "Down", "Left", "Right"]
moves = [(-1,0), (1,0), (0,-1), (0,1)]
GOAL, STEP, OBSTACLE = 10, -1, -5

alpha, gamma, epsilon = 0.1, 0.9, 1.0
Q = np.zeros((size, size, 4))

print("Grid World:")
for r in range(size):
    for c in range(size):
        if (r,c) == start:
            print("S", end=" ")
        elif (r,c) == goal:
            print("G", end=" ")
        elif (r,c) in obstacles:
            print("X", end=" ")
        else:
            print(".", end=" ")
    print()

print("\nActions:", actions)
```

```
print("Rewards: Goal=10, Step=-1, Obstacle=-5")
print("Alpha =", alpha, "Gamma =", gamma, "Epsilon =", epsilon)
print("\nInitial Q-Table:\n", Q)
```

```
= RESTART: E:\ai\lab\expt\test\42a.py
Grid World:
S . . .
. X . .
. . X .
. . . G

Actions: ['Up', 'Down', 'Left', 'Right']
Rewards: Goal=10, Step=-1, Obstacle=-5
Alpha = 0.1 Gamma = 0.9 Epsilon = 1.0

Initial Q-Table:
 [[[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]

 [[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]

 [[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]

 [[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]]
```

B)

```
import numpy as np
import random

# Grid World
N = 4
start, goal = (0, 0), (3, 3)
obstacles = {(1, 1), (2, 2)}
moves = [(-1,0), (1,0), (0,-1), (0,1)]
actions = ["Up", "Down", "Left", "Right"]

# Parameters
alpha, gamma = 0.1, 0.9
epsilon, decay, eps_min = 1.0, 0.98, 0.05
episodes, max_steps = 100, 100

Q = np.zeros((N, N, 4))

def step(s, a):
    r, c = s
    nr, nc = r + moves[a][0], c + moves[a][1]

    if not (0 <= nr < N and 0 <= nc < N):
```

```
        return s, -1, False

    ns = (nr, nc)

    if ns in obstacles:
        return s, -5, False
    if ns == goal:
        return ns, 10, True

    return ns, -1, False

# Q-Learning
for ep in range(episodes):
    state = start

    for _ in range(max_steps):
        action = (random.randrange(4) if random.random() < epsilon
                  else np.argmax(Q[state]))

        next_state, reward, done = step(state, action)

        target = reward if done else reward + gamma * np.max(Q[next_state])
        Q[state[0], state[1], action] += alpha * (
            target - Q[state[0], state[1], action]
        )

        state = next_state
        if done:
            break

    epsilon = max(eps_min, epsilon * decay)

# Final Q-table
print("Final Q-Table:")
for r in range(N):
    for c in range(N):
        print((r, c), np.round(Q[r, c], 2))
```

```
========== RESTART: E:\ai\lab\exp1\test\42b.py ==
Final Q-Table:
(0, 0) [-2.12  1.08 -2.39 -2.55]
(0, 1) [-2.02 -4.41 -2.28 -1.99]
(0, 2) [-1.55 -1.26 -1.9  -1.13]
(0, 3) [-1.05  0.88 -1.27 -1.14]
(1, 0) [-2.46  2.7  -1.7  -5.07]
(1, 1) [0. 0. 0. 0.]
(1, 2) [-1.2  -3.33 -4.79 -0.02]
(1, 3) [-1.11  4.79 -1.08 -0.01]
(2, 0) [-0.99  4.37 -1.1   0.64]
(2, 1) [-3.17  4.84 -0.92 -3.27]
(2, 2) [0. 0. 0. 0.]
(2, 3) [ 0.7   8.5  -0.68  1.44]
(3, 0) [-0.29  1.65  0.15  6.13]
(3, 1) [0.7  1.11 1.41 7.98]
(3, 2) [ 1.12  5.46  2.24 10.  ]
(3, 3) [0. 0. 0. 0.]
>>
```

C)

```
import numpy as np
import random
import matplotlib.pyplot as plt
N = 4
start, goal = (0, 0), (3, 3)
obstacles = {(1, 1), (2, 2)}

moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
symbols = ["↑", "↓", "←", "→"]
alpha, gamma = 0.1, 0.9
epsilon, decay, eps_min = 1.0, 0.98, 0.05
episodes, max_steps = 100, 100

Q = np.zeros((N, N, 4))
rewards = []
def step(state, action):
    r, c = state
    nr = r + moves[action][0]
    nc = c + moves[action][1]

    if not (0 <= nr < N and 0 <= nc < N):
        return state, -1, False
    next_state = (nr, nc)

    if next_state in obstacles:
        return state, -5, False

    if next_state == goal:
        return next_state, 10, True

    return next_state, -1, False
```

```
for episode in range(episodes):
    state = start
    total_reward = 0

    for _ in range(max_steps):
        if random.random() < epsilon:
            action = random.randrange(4)
        else:
            action = np.argmax(Q[state])

        next_state, reward, done = step(state, action)
        target = reward if done else (
            reward + gamma * np.max(Q[next_state])
        )

        Q[state[0], state[1], action] += alpha * (
            target - Q[state[0], state[1], action]
        )
        state = next_state
        total_reward += reward

        if done:
            break
    rewards.append(total_reward)
    epsilon = max(eps_min, epsilon * decay)
print("\nFINAL LEARNED POLICY")
print("----------")
for r in range(N):
    for c in range(N):
        state = (r, c)
        if state == start:
            symbol = "S"
        elif state == goal:
            symbol = "G"
        elif state in obstacles:
            symbol = "X"
        else:
            symbol = symbols[np.argmax(Q[state])]
        print(symbol, end=" ")
    print()
print("\nFINAL Q-TABLE")
print("----------")
for r in range(N):
    for c in range(N):
        print(
            f"State ({r},{c}):",
            np.round(Q[r, c], 2)
        )

window = 10
average = np.convolve(
    rewards,
```

```
    np.ones(window) / window,
    mode="valid"
)
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, episodes + 1), rewards)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Q-Learning Reward")
plt.grid()
plt.subplot(1, 2, 2)
plt.plot(range(window, episodes + 1), average)
plt.xlabel("Episode")
plt.ylabel("Average Reward")
plt.title("Q-Learning Convergence")
plt.grid()
plt.tight_layout()
plt.show()
```

```
FINAL LEARNED POLICY
--------------------
S → ↓ ↓
↓ X → ↓
↓ ↓ X ↓
→ → → G

FINAL Q-TABLE
--------------------
State (0,0): [-2.51 -2.31 -2.13  0.72]
State (0,1): [-0.93 -5.2  -2.05  2.47]
State (0,2): [-1.01  4.26 -1.27 -0.05]
State (0,3): [-0.94  3.64 -1.35 -0.89]
State (1,0): [-1.95 -1.41 -1.79 -5.15]
State (1,1): [0. 0. 0. 0.]
State (1,2): [-0.41 -2.2  -2.24  6.09]
State (1,3): [-0.51  7.97  1.61  2.43]
State (2,0): [-1.62  0.17 -1.11 -0.88]
State (2,1): [-3.19  1.44 -0.47 -3.16]
State (2,2): [0. 0. 0. 0.]
State (2,3): [ 3.29 10.   -0.36  1.77]
State (3,0): [-0.83 -0.65 -0.86  2.69]
State (3,1): [-0.47  1.61 -0.77  6.27]
State (3,2): [-0.19  2.29  0.57  9.42]
State (3,3): [0. 0. 0. 0.]
```